

CS 180: Introduction to Artificial Intelligence

ME 3 - Artificial Neural Network

ME 4 - Support Vector Machine

Introduction

This exercise concerns two different discriminative classifiers: ANN and SVM. For this, you do not need to code for the ANN and SVM themselves. You will be using library implementation of these classifiers. Mainly, the goal of this exercise is to search for the optimal parameters and to make a comparative analysis of the two classifiers for face recognition.

Dataset

The dataset can be downloaded from our Dropbox link. The dataset consists of face images of 20 subjects with varying poses and scale. Each image is encoded as a pgm file. You need to utilize an external library to read these files in constructing the SVM. For Python, you may use the Python Imaging Library(PIL). In constructing the ANN, you may use the built-in functions for reading pgm files. (Take note that the Python function for reading pgm images in the provided code for ANN is not be compatible with the LibSVM.)

The dataset is already divided into training and test sets. Refer to the files *train.list*, *test1.list*, and *test2.list*. The corresponding labels can be found in the files *trainlabel.list*, *testlabel1.list*, and *testlabel2.list*.

Forming the Feature Vector

For each image, form the feature vector by treating each pixel as an attribute. Next, augment the pixel rows with each other. For example, let the matrix below represent the intensity values of the pixels of an image.

$$I = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

The feature vector f for image I is as follows:

$$f = [a \quad b \quad c \quad d \quad e \quad f \quad g \quad h \quad i]$$

ANN

For the Artificial Neural Network, create a three-layer network with the following specifications:

- The input layer should contain one node for each feature.

- The output layer should have one node for each class.

Train and test multiple ANNs, varying the following parameters:

- number of hidden nodes (20 - 500)
- learning and momentum rates
- maximum iterations/epochs (100 - 1000)

Take note of the error per epoch per configuration as the basis for your analysis later on.

The code provided for ANN is a modification of the code by Jeff Shufelt of Carnegie Mellon University. A Python wrapper for the following functions had been made. You may read the source and header files of the C implementation to access low-level functions.

- `read_dataset(filename)`
- `read_label(filename)`
- `create_neural_net(input, hidden, output)`
- `train(net, trainlist, trainlabel, testlist, testlabel, epochs, learning_rate, momentum)`

Creating an ANN

The following shows how to create an ANN.

```
import ann

#create a neural net
input = 28*28
hidden = 100
output = 20
nn = ann.create_neural_net(input, hidden, output)
```

Loading the Dataset

If you use the code provided for ANN, you can use some auxiliary functions for loading datasets and labels.

```
data = ann.read_dataset('train.list')
label = ann.read_label('trainlabel.list')
```

Training and Testing an ANN

The following shows how to train and test an ANN using the code provided.

```
epoch = 100
learning = 0.3
momentum = 0.3
print_errors = 1
train_error_file = 'train_error'
test_error_file = 'test_error'
ann.train(nn, training, traininglabel, testdata,
          testlabel, epoch, learning, momentum,
          print_errors, train_error_file,
          test_error_file)
```

Output of the ANN

The ANN program outputs per epoch a 6-tuple.

```
<ep> <del> <tr_acc> <tr_err> <test_acc> <test_err>
```

where:

ep - epoch number

del - sum of errors of all hidden and output neurons.

tr_acc - percentage of correctly classified training instances

tr_err - sum of squared errors for the training instances

test_acc - percentage of correctly classified test instances

test_err - sum of squared errors for the test instances

SVM

For SVM, construct a multi-class SVM by creating a C-SVC.

Train and test using multiple multi-class SVM, varying the following parameters:

- kernel type (linear, polynomial, radial basis, sigmoid)
- degree of kernel function (1 - 10) [applicable only for polynomial kernel]
- gamma ($0 < \text{gamma} \leq 1$, use 0.1 increments) [applicable to all kernels except linear]
- coef0 (0 - 10) [applicable only for polynomial and sigmoid kernel]

Creating and Training an SVM

The following describes how to create a C-SVC using LibSVM.

```
#the third argument of the function refers
#to the parameters you need to modify for the SVM.
#see the README file of the LibSVM for more info
SVM = svm_train(labels, data, '-q')
```

Testing using SVM

The following shows how to test an SVM.

```
classification,accuracy,val =
    svm_predict(label, test_data, SVM)
```

Testing

For testing, use the remaining images not used in training. Prepare the test images in the same manner as the training images: form feature vector by squashing the image into a row vector.

Output

The output of this machine exercise will be in the form of a report. Write a report, with at least one page, answering the following questions. Base your answers from your experiments.

1. Fixing some parameters while varying one, what is the overall effect to the accuracy of each classifier?
2. Which of the two classifiers train faster? Which of the two test faster?
3. Overall, was it a good idea to treat one pixel as one feature? Why or why not?
4. Compare the best and worst accuracy of the two classifiers. Which is better suited for this particular task?
5. For the neural network, at what epoch did it reach its maximum training accuracy?
6. For each classifier, what is the maximum and minimum discrepancy between training and test accuracy? What do these values tell about the built classifiers? (Hint: data overfitting).

Submission

- Deadline of submission is on 29 March 2014 11:59PM.
- Submit the following deliverables to **kedelaspe-nas@up.edu.ph** with the email subject **CS 180 ME3-4: <surnames>**.
 - PDF report
 - Source code
 - Graph of the accuracy of the two classifiers using the determined optimal parameters

Notes

- You may work with a partner.
- Rules against cheating and plagiarism will be strictly observed.
- You may use other libraries for some subtasks in training and testing. Moreover, you may use libraries other than LibSVM and the code provided for ANN, if you find some difficulty in using them. Just make sure that upon submission, all libraries used are stated with respective download links in the email.

- You might encounter a lot of problems in installation and running the suggested libraries, especially the code for ANN (I only tested it on a 32-bit Debian virtual machine). Compiling the ANN library requires swig, and a C compiler. As a last resort, you can ask me for the configured VM with the libraries.

References

- [1] Chang, Chih-Chung and Lin, Chih-Jen. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*. Vol. 2. Issue 3. 2011. p.27. Software available online: <http://www.csie.ntu.edu.tw/~cjlin/libsvm>
- [2] CMU Face Images Data Set. UCI Machine Learning Repository. Available online: <https://archive.ics.uci.edu/ml/datasets/CMU+Face+Images>
- [3] Unmodified ANN code Available online: <http://www.cs.cmu.edu/afs/cs.cmu.edu/project/theo-8/faceimages/code/>